

Sections 5.1 – 5.7

Routing Algorithms · OSPF · BGP · SDN
ICMP · Network Management

Kurose & Ross, 8th Ed.



Chapter 5 — Control Plane Roadmap

Introduction

Control Plane: the network-wide intelligence that builds and maintains forwarding tables

5.1 Two Control Plane Approaches

Per-router control (OSPF/BGP): routing algo inside every router — distributed.

SDN: remote controller computes and distributes tables — programmable.

5.3 OSPF — Intra-AS Routing

Open Shortest Path First. Link-state flooding within an AS. Dijkstra locally. Hierarchical areas for scale.

5.5 SDN Control Plane

Controller: southbound (OpenFlow) · state mgmt · northbound (REST API). Apps: routing, firewall, load balancing. Google B4.

5.2 Routing Algorithms

Link-State (LS): global topology + Dijkstra.

Distance-Vector (DV): local view + Bellman-Ford.

Count-to-infinity problem and mitigations.

5.4 BGP — Inter-AS Routing

The Internet's glue. Path-vector over TCP. eBGP/iBGP. AS-PATH loop detection. POLICY over performance.

5.6 ICMP · Network Management

ICMP: Ping + Traceroute.

SNMP: query/set MIB objects (monitoring).

NETCONF/YANG: transactional config at scale.

5.1 — Two Control Plane Approaches

Section 5.1: Introduction

How are forwarding tables built? Two fundamentally different architectural philosophies.

Per-Router Control (Traditional)

A ROUTING ALGORITHM runs inside EACH AND EVERY router.

Routing components communicate with each other — exchanging topology and link cost information.

Each router independently computes its own forwarding table.

Data plane and control plane are MONOLITHIC — both in every box.

Key property: DISTRIBUTED — no single point of failure.

Used by: OSPF (intra-AS) · BGP (inter-AS)

Dominant paradigm since ARPANET.

Logically Centralized Control (SDN)

A physically separate REMOTE CONTROLLER computes forwarding tables and distributes them to all routers.

Routers become "dumb" forwarding devices — no routing algorithms inside.

Controller has a GLOBAL VIEW of the network.

Communication via OpenFlow protocol (TCP port 6653).

Key property: PROGRAMMABLE — change network behavior by modifying software, not hardware.

Used by: Google B4 · AT&T · Data center fabrics.

The modern SDN paradigm.

Both produce forwarding tables at routers — what differs is WHERE and HOW those tables are computed

5.2 — Routing Algorithms: Finding Least-Cost Paths

Section 5.2: Routing Algorithms

Goal: find the LEAST-COST PATH between a source and all destinations — used to build forwarding tables

Network as a GRAPH: Nodes = routers · Edges = links · Edge weights = costs (bandwidth, delay, or 1 hop)

Link-State (LS) Routing

GLOBAL KNOWLEDGE: every router knows the complete network topology.

How: each router BROADCASTS (floods) its link costs to ALL others → everyone builds the same complete map.

Algorithm: each router independently runs DIJKSTRA'S SHORTEST PATH on its local copy of the global map.

Result: globally optimal paths from day one.

Used by: OSPF, IS-IS

Distance-Vector (DV) Routing

LOCAL KNOWLEDGE ONLY: each router knows only direct neighbor costs + neighbors' estimates.

Algorithm: BELLMAN-FORD — iterative, asynchronous, distributed:

$$d_x(y) = \min_v \{ \text{cost}(x,v) + d_v(y) \}$$

"cheapest path = cheapest neighbor + that neighbor's estimate"

Routers exchange vectors with neighbors, update, repeat until no changes (convergence).

Used by: RIP · BGP (path-vector variant)

Despite different approaches, BOTH algorithms converge to the same globally optimal paths when the network is stable

Link-State Routing — Dijkstra's Algorithm

Section 5.2.1

LS: flood link costs to all → each router runs Dijkstra locally on the complete map

Dijkstra's Algorithm — Concept

ITERATIVE: at each step, commit to the shortest path to one more node — building outward from the source like an expanding circle.

1. Start: known set $N' = \{\text{source}\}$. Initial estimates = direct link costs.
2. Find the node NOT in N' with the SMALLEST known cost.
3. Add it to N' . Update cost estimates for its neighbors.
4. Repeat until ALL nodes are in N' .

Complexity: $O(n^2)$ for n routers — or $O(n \log n)$ with a heap.

Result: a forwarding table for the source — one next-hop per destination.

All routers run THE SAME algorithm on THE SAME map → all compute the same optimal paths simultaneously.

Properties and Practical Notes

ADVANTAGES:

- Fast convergence once topology is distributed
- Globally optimal paths
- Robust: one router's bad info mostly affects local area only

DISADVANTAGES:

- Flooding overhead: each router broadcasts to ALL others
- All-to-all communication requires significant bandwidth in large networks

OSCILLATION RISK:

If link costs reflect traffic load, all routers simultaneously reroute away from congested links → they pick the same alternative → that link now congests → oscillation.

Solution: routers randomize the timing of link-state advertisements.

Flooding + local Dijkstra = each router independently computes the globally optimal routing table

Distance-Vector Routing — Bellman-Ford and Count-to-Infinity

Section 5.2.2

DV: no global map — iteratively refine estimates using only neighbors' information

How DV Routing Works

INITIALIZATION: each router knows only costs to direct neighbors.

BELLMAN-FORD (at router x, for each destination y):

$$d_x(y) = \min_v \{ \text{cost}(x,v) + d_v(y) \}$$

"Best path = best neighbor + that neighbor's best path"

PROCESS:

1. Send my distance vector to all neighbors.
2. Receive neighbors' updated vectors.
3. Recompute all entries using Bellman-Ford.
4. If any entry changed, send updated vector to neighbors.
5. Repeat until no more changes → CONVERGENCE.

Self-terminating — no explicit stop signal.

Asynchronous — routers don't operate in lock-step.

Distributed — each router computes only its own entries.

Count-to-Infinity Problem

SCENARIO: nodes x, y, z in a line. x—y link cost increases dramatically or link fails.

BAD NEWS TRAVELS SLOWLY:

y thinks: "I can reach x via z (cost z+1)."

z thinks: "I can reach x via y (cost y+1)."

→ Both counting up toward infinity in a loop.

Routers keep exchanging optimistic (wrong) estimates, slowly counting up before agreeing the destination is unreachable. Causes temporary ROUTING LOOPS.

MITIGATIONS:

- POISON REVERSE: advertise a route as ∞ back to the neighbor you learned it from. Eliminates 2-node loops.
- TRIGGERED UPDATES: immediately send updates when a route changes.
- SPLIT HORIZON: never advertise a route back through the interface you learned it on.

Good news propagates fast in DV · Bad news propagates slowly (count-to-infinity) · LS (OSPF) doesn't have this problem

Link-State vs. Distance-Vector — Complete Comparison

Section 5.2

Both converge to optimal paths — but differ in approach, properties, and failure behavior

Dimension	Link-State (LS)	Distance-Vector (DV)
Information scope	GLOBAL — complete topology known to all	LOCAL — only neighbor costs + estimates
Computation	Dijkstra (centralized at each router)	Bellman-Ford (distributed among neighbors)
Convergence speed	Fast after flooding completes	Slower — bad news travels slowly
Key problem	Oscillation with load-sensitive costs	Count-to-Infinity — loops on bad news
Failure robustness	Incorrect LS broadcast → mostly local impact	Incorrect DV estimate → error propagates widely
Communication overhead	Flooding grows with network size	Exchange only with direct neighbors
Real protocol	OSPF · IS-IS	RIP · BGP (path-vector variant)

Internet uses BOTH: LS inside each AS (OSPF) · DV-inspired path-vector between ASs (BGP)

5.3 — Autonomous Systems and OSPF

Section 5.3: Intra-AS Routing

The Internet is divided into Autonomous Systems — two routing tiers: intra-AS and inter-AS

Why Autonomous Systems?

An AUTONOMOUS SYSTEM (AS) is a group of routers under the same administrative control. Typically one ISP = one or more ASs. Each AS has a globally unique ASN assigned by ICANN.

TWO ROUTING TIERS:

INTRA-AS: routing WITHIN a single AS

→ Same admin control, optimize performance

INTER-AS: routing BETWEEN different ASs

→ Multiple admins, policy matters

WHY HIERARCHY?

1. SCALE: one routing algorithm across millions of routers is computationally infeasible. Breaking into ASs makes it manageable.

2. AUTONOMY: each organization wants to run its own routing policy internally, independently of outsiders.

OSPF — Open Shortest Path First

Dominant INTRA-AS routing protocol. [RFC 2328]

OSPF is a LINK-STATE protocol:

- Each router floods link-state info to EVERY router in the AS.
- Every router builds a COMPLETE MAP of the AS.
- Each router independently runs DIJKSTRA to find shortest paths to all subnets.

Key features:

- AUTHENTICATION — MD5 prevents bogus routing info injection.
- MULTIPLE EQUAL-COST PATHS — load balance across parallel links.
- HIERARCHICAL AREAS — AS divided into areas; flooding stays within each area. Backbone area 0 connects all others.

OSPF areas: confine link-state flooding to a subset of routers → dramatically reduces overhead in large networks

5.4 — BGP: The Internet's Glue

Section 5.4: Routing Among the ISPs

BGP — Border Gateway Protocol — the SINGLE protocol that holds ~100,000 autonomous systems together

Why BGP Is Necessary

OSPF handles routing WITHIN an AS. Packets crossing MULTIPLE ASs — from Bucharest to Singapore — need a different protocol.

OSPF cannot be used across ASs: each AS hides its internal topology and makes its own policy decisions.

BGP is universal: EVERY AS on the Internet runs it.

BGP routes PREFIXES (e.g., 200.23.18.0/23), not individual addresses. A BGP forwarding entry:
(prefix → outgoing interface)

BGP's Two Core Tasks

1. ADVERTISE REACHABILITY:

Allow subnets to announce existence to the rest of the Internet. "I am subnet x — I am here and reachable."

2. DETERMINE BEST ROUTES:

A router may learn about a prefix from many paths. BGP selects the BEST route based on POLICY and path information.

Result: every router has forwarding table entries for external prefixes → a packet can travel from any host on Earth to any other.

BGP sessions: semi-permanent TCP connections (port 179) between border routers

eBGP — External BGP

Session between gateway routers in DIFFERENT ASs.
Purpose: exchange reachability information between ASs.
One eBGP connection per inter-AS physical link.

iBGP — Internal BGP

Session between routers WITHIN THE SAME AS.
Purpose: propagate externally learned routes to ALL internal routers.
Full mesh of TCP connections within the AS.

BGP Route Attributes — AS-PATH, NEXT-HOP, Hot Potato Routing

Section 5.4.2 / 5.4.3

BGP advertises routes with ATTRIBUTES that encode path information and enable policy decisions

AS-PATH Attribute

Records the SEQUENCE of ASs a route advertisement has traversed.

Route propagation example:

AS3 has prefix x.

AS3 → AS2: advertises "AS3 x"

AS2 → AS1: advertises "AS2 AS3 x"

AS1 learns: x reachable via AS2, then AS3.

TWO CRITICAL USES:

1. LOOP DETECTION: if AS sees its own ASN in the path, it discards the advertisement.

2. ROUTE SELECTION: shorter AS-PATHs are generally preferred.

NEXT-HOP Attribute

IP address of the FIRST ROUTER on the path to the advertised prefix — the entry point into the advertising AS.

A router must use INTRA-AS routing (OSPF) to find the cheapest path to the NEXT-HOP router, and THEN to the destination prefix.

HOT POTATO ROUTING:

When multiple BGP paths exist for a prefix, choose the path whose NEXT-HOP has the LOWEST INTRA-AS COST.

"Get the packet out of my AS as cheaply as possible — like passing a hot potato!"

Ignores total end-to-end cost — only cares about cost WITHIN your AS.

BGP Route Selection Order: 1) Local preference (policy) 2) Shortest AS-PATH 3) Closest NEXT-HOP (hot potato) 4) Router ID (tiebreaker)

Why This Order?

LOCAL PREFERENCE always wins — the network admin explicitly specifies which routes the AS prefers (policy over everything).

SHORTEST AS-PATH next — fewer AS hops is generally better.

HOT POTATO last among performance criteria — minimize cost within your own network before caring about outside.

This ordering encodes the fundamental principle: POLICY beats PERFORMANCE in inter-AS routing.

BGP Routing Policy — Commercial Relationships Drive Routing

Section 5.4.5

BGP's most important insight: POLICY dominates over performance — commercial relationships determine routes

Commercial Relationship Types

CUSTOMER–PROVIDER:

Customer AS pays Provider AS for transit.
Provider carries customer traffic to/from the world.

PEER–PEER (settlement-free peering):

Two ASs of similar size exchange traffic directly.
Neither pays the other.

THE FUNDAMENTAL RULE:

Traffic across an ISP's backbone must have a SOURCE or DESTINATION (or both) in a network that is a CUSTOMER of that ISP.
Otherwise: the ISP is giving away free transit — carrying traffic between peers without compensation.

Policy in Practice — Advertisement Rules

B and C are backbone ISPs (peers). A is a customer of B. D is a customer of C.

What B WILL do:

- Advertise A's routes to C → A reaches the world
- Advertise C's routes to A → A can reach D

What B will NOT do:

· Advertise C's routes through to a 4th AS via A — that would make B a free transit between peers and A.

ADVERTISEMENT RULES:

- To customer: always (they pay you)
- To provider: yes (you're paying them)
- To peer: only customer routes, NOT peer routes

BGP encodes commercial relationships into routing — it is a POLICY protocol, not a performance optimization protocol

Why Different Protocols? OSPF vs. BGP

Section 5.4

Intra-AS and inter-AS routing have different goals — they need fundamentally different protocols

POLICY

OSPF (intra-AS)

Single admin domain.
Routers trust each other.
Optimize for least cost, shortest path, maximum throughput.
Policy issues minimal.

BGP (inter-AS)

Multiple admins with different business interests.
Commercial relationships govern route advertisement.
Performance is secondary to policy.

SCALE

OSPF (intra-AS)

Runs within one AS — hundreds to a few thousand routers.
Dijkstra on complete topology is feasible.
Hierarchical areas handle larger ASs.

BGP (inter-AS)

~100,000 ASs, millions of IP prefixes globally.
No single algorithm could handle this.
ASs hide internal topology.
BGP works at AS-level granularity.

AUTONOMY

OSPF (intra-AS)

All routers in the AS run the SAME protocol, share the SAME link-state database.
No administrative independence needed.

BGP (inter-AS)

Each AS can run ANY intra-AS protocol internally.
BGP only requires agreement at AS boundaries.
An AS can change internal routing freely.

BGP is what makes the Internet a NETWORK OF NETWORKS — routing among 100,000 independent ASs with no central authority

5.5 — SDN Control Plane Architecture

Section 5.5: The SDN Control Plane

SDN: separate control plane from data plane — intelligence in software, switching in commodity hardware

SDN Controller — Three Layers

TOP Northbound API

Interface for NETWORK-CONTROL APPLICATIONS.
REST API or event callbacks.
Apps read state and write flow tables.

MID State Management

Global view: all links, hosts, switches, flow tables.
The controller's shared database — the brain.

BOT Southbound (OpenFlow)

Controller → switch: add/remove flow rules.
Switch → controller: link failures, unmatched packets.

Network-Control Applications

Routing: Runs Dijkstra, installs forwarding rules

Access Control: Blocks/allows flows — distributed firewall

Load Balancer: Distributes connections across servers

Traffic Engineering: Optimizes global bandwidth allocation

Why SDN Matters

VENDOR INDEPENDENCE: switches → commodity hardware. Intelligence in software.
INNOVATION: new routing algorithm = a Python app, no new RFC needed.
PROGRAMMABILITY: reconfigure the entire network in software in real time.
Deployments: Google B4 · Microsoft, Facebook · AT&T (core).

SDN in Action — OpenFlow and Link-Failure Response

Section 5.5.2 / 5.5.3

OpenFlow (TCP port 6653): the dominant southbound protocol for controller–switch communication

Key OpenFlow Messages

Controller → Switch

Modify-State:	Add/remove/modify flow table entries
Send-Packet:	Inject a specific packet out of a port
Read-State:	Query flow table counters and statistics

Switch → Controller

Packet-In:	Packet matched no flow rule — what to do?
Port-Status:	A link/port changed state (up/down)
Flow-Removed:	A flow table entry has expired

Example: Link Failure Response

Scenario: link between switch s1 and s2 goes DOWN

- 1 s1 detects link failure → sends PORT-STATUS message to SDN controller.
- 2 Controller updates link-state database (global view) and notifies apps.
- 3 Controller notifies the routing application that topology changed.
- 4 Routing app re-runs Dijkstra on the updated topology graph.
- 5 App sends MODIFY-STATE messages to affected switches (s1, s3, s4).
- 6 Traffic automatically reroutes — no human intervention required.

vs. traditional OSPF: routers gossip among themselves, each independently recomputes — may take many seconds

5.6 — ICMP: The Internet Control Message Protocol

Section 5.6: ICMP

ICMP: how routers and hosts communicate network-layer errors and diagnostics to each other

What Is ICMP?

ICMP [RFC 792]: used by hosts and routers to communicate NETWORK-LAYER INFORMATION — errors and diagnostics.

ICMP messages travel INSIDE IP DATAGRAMS (protocol field = 1). It sits "just above IP."

Key message types:

- Type 0: Echo Reply (Ping response)
- Type 3: Destination Unreachable
- Type 8: Echo Request (Ping)
- Type 11: TTL Expired ← powers Traceroute!
- Type 12: Bad IP header

PING — Is the Host Alive?

Source sends ICMP Echo Requests (type 8).
Destination replies with Echo Replies (type 0).
Source measures ROUND-TRIP TIME.
"Is that host alive and reachable?"

TRACEROUTE — Mapping the Path Hop by Hop

- Source sends probe packets with TTL = 1, 2, 3, ...
- TTL = 1: first router decrements to 0, drops packet, sends ICMP "TTL Expired" (type 11) — revealing its IP address.
- TTL = 2: second router expires and reveals itself. And so on.
- THREE probes per TTL value → three RTT measurements per hop.
- Continue until destination reached → full path and per-hop RTT revealed.
- Invented ~1988, still the most powerful network diagnostic tool.

RTT to a hop can DECREASE compared to the previous hop — variable queuing means RTT is NOT monotonically increasing

Key Conceptual Insights from Chapter 5

SUMMARY

Summary

Five big ideas that define the network layer control plane

1. Scale through hierarchy

The Internet is too large for one routing algorithm. Two tiers — OSPF intra-AS + BGP inter-AS — make global routing computationally and administratively manageable.

2. Policy over performance in inter-AS routing

BGP routes based on commercial relationships, not shortest paths. A provider ISP will NOT carry free transit between peers. Business logic drives routing.

3. Distributed vs. centralized intelligence

Per-router control (OSPF/BGP): distributed, robust, proven. SDN: centralized, programmable, globally optimal. Each has advantages for different contexts.

4. Bad news travels slowly in DV routing

Count-to-infinity: when a link fails, DV routers may loop indefinitely updating each other with wrong estimates. LS (OSPF) avoids this with Dijkstra's global view.

5. ICMP is the network's feedback mechanism

Without ICMP, IP would be completely silent on errors. ICMP provides the feedback loop that makes network diagnosis possible — Ping and Traceroute are its most visible tools.

Summary

Chapter 5 — Network Layer: Control Plane

- 5.1 Two approaches: Per-router control (monolithic, distributed — OSPF/BGP) vs. SDN logically centralized (programmable — remote controller computes and distributes forwarding tables).
- 5.2 Routing algorithms: Link-State (global knowledge + Dijkstra → OSPF) vs. Distance-Vector (local + Bellman-Ford, count-to-infinity problem → RIP/BGP). Both converge to optimal paths.
- 5.3 OSPF (intra-AS): link-state flooding within AS + Dijkstra. Key features: authentication, multiple equal-cost paths, hierarchical areas to contain flooding overhead at scale.
- 5.4 BGP (inter-AS — the Internet's glue): path-vector over TCP (port 179). eBGP (between ASs) + iBGP (within AS). AS-PATH prevents loops. NEXT-HOP + hot potato. POLICY over performance — commercial relationships drive routing.
- 5.4 Why OSPF ≠ BGP: intra-AS → optimize performance, single admin, manageable scale. Inter-AS → policy dominates, multiple admins, ~70,000 ASs, millions of prefixes globally.
- 5.5 SDN control plane: controller layers — southbound (OpenFlow: flow rules, port-status) · state management (global view) · northbound (REST API for control apps). Google B4, data centers.
- 5.5 SDN applications: routing, access control, load balancing, traffic engineering. Key benefit: network programmability — new behavior = software, not hardware changes.
- 5.6 ICMP: error reporting inside IP datagrams (protocol field = 1). Ping: Echo Request/Reply (type 8/0). Traceroute: TTL Expired messages (type 11) reveal each hop's identity and RTT.
- 5.7 SNMP: query (GetRequest) / configure (SetRequest) / trap — monitor MIB objects device by device. Strong for monitoring, limited for configuration at scale.
- 5.7 NETCONF/YANG: YANG models configuration data; NETCONF applies changes transactionally across multiple devices atomically — foundation for network automation and infrastructure-as-code.